

Prompt Injection as Pipeline Architecture

How `scripts/gen/osp_generate.py` steers Jac code generation

Technical report — `jac_llm_data`

September 17, 2026

1 Executive summary

`scripts/gen/osp_generate.py` is the unified entrypoint for the OSP (object-spatial programming) dataset generation pipeline: one command routes *idiomize* (issue $\rightarrow$ Jac lift) and *codegen* (problem $\rightarrow$ Jac) jobs across five transports (`pi`, `cursor`, `zen`, `openrouter`, `zai`). Its codegen path imports the system prompt, user-prompt builders, and validation gates from `scripts/gen/osp_luna_generate.py` (lines 88–91), and it is there that the **prompt injection technique** lives. The term is used here in the literal, non-adversarial sense: *the pipeline injects grounding material into the model’s context at runtime*, from files, distilled skills, verified demos, and even from the model’s own compiler errors.

There are two distinct injection sites:

1. **Import-time system-prompt surgery.** At module load, `osp_luna_generate.py` mutates a 3,740-character “floor” system prompt through six operations — an in-place rule *replacement*, two *file splices*, a *verified demo*, a *skill distillation*, *error-code-derived bans*, and a *corpus reference record* — producing a 14,077-character payload. **73% of the final system prompt is injected**, not written.
2. **Error-conditioned hint injection (runtime).** When a generated program fails a gate, `_fix_user()` regex-scrapes the validator error (e.g. `undefined name 'Owner'`) and *injects a targeted repair hint* into the retry prompt, closing a generate–validate–repair loop bounded by `fix_tries` (default 2).

Measured effect, from the code’s own probe (comment at `osp_luna_generate.py:146–149`): ten holdout problems that failed the floor prompt three times each went **0/10 $\rightarrow$ 3/10** once the injected context was added.

A containment property worth noting: dataset records persist only the *raw* problem and the final code in `messages` (`osp_generate.py:199–200`). The injected context is never written into the training transcript; it is versioned instead through metadata fields (`source_prompt_version`, `context_bundle_version`, `validator_version`).

2 Pipeline overview

`osp_generate.py` dispatches on `-task`:

- **idiomize** (`_dispatch_idiomize`, lines 38–79): patches env vars and module globals of `osp_agent_generate.py`, rewrites `sys.argv`, and re-enters that script’s `main()`. The *idiomize* path runs its own, parallel injection: `load_spec_excerpt()` (`osp_agent_generate.py:198–204`) splices `data/osp_examples/GEN_CONTRACT.md` (first 6,000 chars) and `docs/OSP_IDIOMIZE_TASK.md` (first 8,000 chars), and call sites invoke the model with `SYSTEM + "\n" + spec[:4000]` (lines 643, 746).

- **codegen** (`_dispatch_codegen`, lines 82–208): the shared-backend path analyzed below. It selects a transport via `get_backend()` (`scripts/lib/llm_backend.py`), replays the 100 hand-written problems in `data/osp_examples/group_{A..E}.jsonl`, gates each answer, and appends passing records to `data/osp_dataset.jsonl`.

3 Injection site 1: import-time system-prompt surgery

At `osp_luna_generate.py:146–253`, module top level (executed once at `import`, which is why `osp_generate.py` inherits it for free):

```
SYSTEM = SYSTEM.replace(_CEILING, _UNION_NOTE) # rule swap
SYSTEM += "\n\nIDIOMATIZATION CONTRACT ... \n" + _SPEC[:4000] + "\n\n" + _SPEC6
        + "\n\nVERIFIED IDIOMATIC PATTERN ... \n" + _DEM0 + "\n" + _FILTER_WARN
SYSTEM += _SKILL_TIER0 # distilled from the jac-idiomatic skill
SYSTEM += _COMPILER_PITFALLS # E0013 / E0076 / E0002 / E0004
SYSTEM += "\n\nREFERENCE RECORD (from the reference corpus): \n" + _REF_PATH.read_text()[:3500] + "\n" # verified corpus file
```

Each operation has a distinct mechanism and purpose:

Layer	Chars	Mechanism / purpose
SYSTEM literal (“floor prompt”)	3,740	kept in place; output contract, structure rules, reference shape
<code>.replace(_CEILING, _UNION_NOTE)</code>	+182	<i>string surgery on a rule</i> : swaps the “one ability per node type” ceiling for union triggers (can inspect with TypeA TypeB entry)
<code>_SPEC[:4000]</code> splice	4,000	first 4k of <code>docs/OSP_IDIOMIZE_TASK.md</code> injected verbatim
<code>_SPEC6</code> slice	1,705	<i>section-scoped</i> splice: only “## 6. The model task” of the same doc
<code>_DEM0</code> verified demo	1,031	a complete Jac program stated to “run clean on jac 0.36.1”; anchors format and idioms
<code>_FILTER_WARN</code>	122	negative constraint: bans pre-0.36 parenthesized filter syntax from stale training data
<code>_SKILL_TIER0</code> syntax floor	1,013	distilled from the jac-idiomatic skill; brace blocks, semicolons, keyword bans
<code>_COMPILER_PITFALLS</code>	602	mined from real failures: report reserved (E0013), duplicate methods (E0076), bracket balance (E0002), unknown fields (E0004)
REFERENCE RECORD	1,622	a real, passing corpus file (<code>ex_22_hospital_union_staff.jac</code>), capped at 3,500 chars
Injected total	10,337	73% of the final 14,077 chars

Table 1: Import-time injection stack, measured byte budget (`osp prompt -stat` reproduces this table live).

The design intent is explicit in the code comments: the small models “imitate reference structure well but never attempt the signature OSP idioms (union triggers, typed edges) unless shown them, and ... carry pre-0.36 filter syntax from stale training data.” The injection is therefore *exemplar-conditioned*:

show the idiom, ban the stale one, prove the demo compiles, and quote a corpus record that already passed.

4 Injection site 2: error-conditioned repair hints

The second injection is *runtime and data-dependent*. In the per-problem loop (`osp_generate.py:162-183`), every gate failure re-enters the model with `_fix_user(problem, prev_code, last_error)` (`osp_luna_generate.py:263-282`):

```
def _fix_user(problem, code, error):
    m = re.search(r"undefined name ['\"](\w+)", error or "")
    hint = (
        f"\n\nThis error means '{m.group(1)}' is used as an edge type in a "
        "'visit [->:...]' / '[<--:...]' or a '+>:Name:+>' connection but was "
        f"never declared. Fix: add 'edge {m.group(1)} {{ }}' at module level ..."
        if m else ""
    )
    return f"### Problem\n...\n### Validation error\n{error}\n{hint}\n..."
```

The loop, with its gates:

1. `extract_jac()` — regex-extracts the single "jac fenced block (a miss is itself a repairable failure).
2. `jac_check()` (line 320) — `jac check` (syntax/types), then `jac run` in a temp dir, then *stdout must be non-empty*: a silent pass is rejected, because the reference corpus always prints demo output.
3. `osp_contract()` (line 365) — eight regex gates: top-level docstring; no `Any/object`; typed `has` fields; `visit` after a typed entry; `with entry` demo block; a `spawn` call; the self-referential ternary trap (`x = "a" if c else x`); no Python `__main__` idiom; and no `visit` referencing an undefined name.
4. On failure: the error is logged to `data/osp_dataset_failures.jsonl` together with the broken code — explicitly kept as “raw material for repair-training pairs” — and the *injected-hint repair prompt* fires, at most `fix_tries` (2) times.

5 How it works, end to end

6 Why the technique works

- **Negative constraints encode compiler reality, not taste.** Every ban maps to a concrete failure mode (`report` → E0013; duplicate methods → E0076; filter syntax → removed in Jac 0.36). The prompt and the gate agree.
- **Exemplars beat descriptions for small models.** The verified demo and the reference record anchor output *shape* (single fence, docstring first, `with entry` last) that the regex gates then enforce.
- **The loop makes the injection adaptive.** The static injection covers systematic gaps (idioms, stale syntax); the runtime hint injection covers per-instance failures, converting validator output — the one signal guaranteed to be fresh — into prompt content.

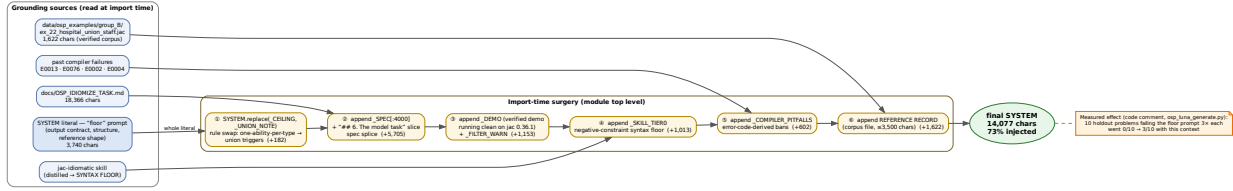


Figure 1: Import-time injection stack. Six operations transform the 3,740-char floor prompt into a 14,077-char grounded payload; 73% of the final system prompt is injected from files, skills, demos, and error history.

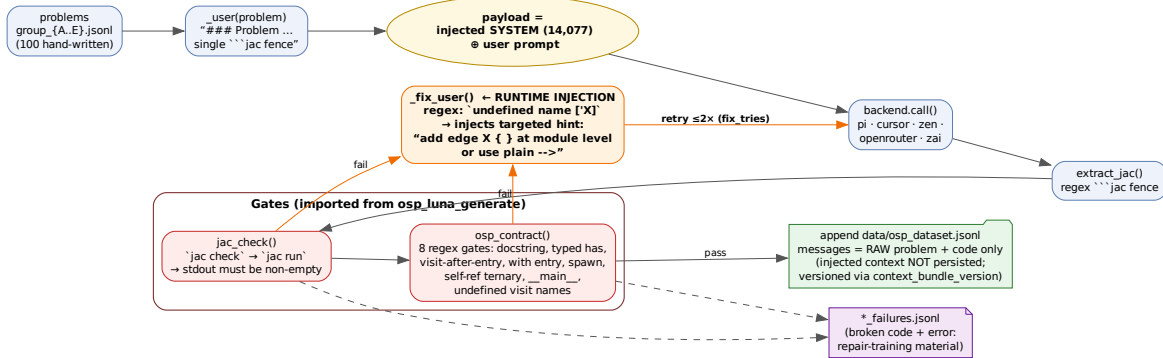


Figure 2: Runtime loop. The injected SYSTEM rides every call; gate failures feed back through `_fix_user`, which regex-scrapes the compiler error and injects a targeted hint — a second, data-conditioned injection site.

- **Grounding is measured.** The code comment records the A/B: 0/10 $\rightarrow$ 3/10 on ten 3 $\times$ -failed holdouts. All 100 reference records are stated to pass the same gates, so the bar is consistent across generators.

Risks. (i) *Silent prompt drift*: because 73% of the prompt is read from files at import time, editing `docs/OSP_IDIOMIZE_TASK.md` changes generation behavior without touching the script; the `context_bundle_version` metadata is the only provenance. (ii) *Ordering sensitivity*: layers append in fixed order (`.replace` $\rightarrow$ `splice` $\rightarrow$ `demo` $\rightarrow$ `bans`), so reordering changes precedence. (iii) The 3,500-char cap on the reference record truncates silently mid-file.

7 The bash tool: `scripts/osp`

The report’s companion deliverable wraps the Python entrypoint as a first-class shell command (`scripts/osp`, `bash`, self-contained):

```
osp codegen --backend pi --model gpt-5.6-luna --limit 10 # run a wave
osp dry codegen --limit 5 # plan, no API calls
osp idiomize --batch 35 --flow jac-only # issue-lift wave
```

```

osp prompt          # dump the assembled (injected) system prompt
osp prompt --stat   # live per-layer injection budget (Table 1)
osp stats           # dataset + failure counters, by generator
osp completion      # emit bash completion; osp install -> ~/.local/bin

```

`osp prompt -stat` makes the injection *inspectable from the shell* — the byte budget of Table 1 is recomputed live from the same module the pipeline uses, so any edit to the spec or corpus files is immediately visible as a changed injection budget.

8 Reproduction

```

python3 - <<'EOF'          # measure the injection stack
import sys; sys.path.insert(0, 'scripts/gen')
import osp_luna_generate as L
print(len(L.SYSTEM))        # 14077
EOF
dot -Tpdf docs/figures/osp_injection_stack.dot -o docs/figures/osp_injection_stack.pdf
pdflatex -interaction=nonstopmode docs/osp_prompt_injection_report.tex # this PDF
./scripts/osp prompt --stat && ./scripts/osp dry codegen --limit 3

```